

001

002

003

004

005

006

007

008

009

010

011

012

013

014

015

016

017

018

019

020

021

022

Supplementary Materials for “Efficient
Vision-Language Models for Resource-Constrained
Edge Deployment: Top-N Robot Selection and
Multimodal Reasoning”

001

002

003

004

005

006

007

008

009

010

011

012

013

014

015

016

017

018

019

020

021

022

005

006

Anonymous ECCV 2026 Submission

005

006

Paper ID #6999

006

007

008

Appendix

007

008

009

1 Robot Selection Dataset Statistics

008

009

010

011

012

013

014

015

016

017

018

019

020

021

022

The Robot Selection dataset contains 500 samples designed to evaluate multi-label reasoning over robotic platforms. Scenarios were collected from common robotics use cases such as aerial monitoring, underwater inspection, indoor navigation, and uneven terrain mobility, and were further expanded using synthetic prompts generated from multiple large language models to increase environmental and task diversity. The generated outputs were manually filtered to remove duplicates and ambiguous descriptions, after which each scenario was annotated with one or more applicable robot categories among five classes. The task is formulated as multi-label classification where a scenario may correspond to multiple suitable robot types. The dataset is split into train, validation, and test partitions using stratified sampling based on label co-occurrence, and evaluation follows standard multi-label metrics including precision, recall, and F1 score.

009

010

011

012

013

014

015

016

017

018

019

020

021

022

500 total samples; multi-label task (up to all 5 robot classes per sample). Splits are stratified by class co-occurrence.

Table 1: Split distribution and per-class label counts.

	Train	Val	Test	Total
<i>Samples</i>	331	103	66	500
Drone	148	53	25	226
Underwater Robot	41	13	10	64
Humanoid	166	46	31	243
Robot with Wheels	54	15	12	81
Robot with Legs	166	45	36	247

2 Episodic Memory: Modes, Ablation, and Per-Class Analysis

OFF: no memory. **READ_ONLY**: memory pre-populated offline from the training split, frozen at eval time (no test-time writes; eliminates any risk of label leakage). **ONLINE_UPDATE**: read and write during evaluation, simulating live edge deployment where the device continuously refines its experiential cache. Stage 5 knowledge consolidation was *not* applied for any result in the main text.

Table 2: Memory mode ablation across all tasks. Robot Sel.: 250 samples; VERI: 200; GEO: 3,001.

Mode	Top-N Robot Selection			VERI		GEO	
	Ex.Acc	Mac-F1	Jaccard	Acc	UnderRx↓	Acc	Time(s)
OFF	0.088	0.229	0.194	0.481	0.285	0.210	2755.8
READ_ONLY	0.116	0.248	0.219	0.487	0.308	0.194	3125.7
ONLINE_UPDATE	0.074	0.227	0.170	0.496	0.215	0.203	3403.8

READ_ONLY leads on robot selection, confirming that the memory’s primary value is *structured retrieval* rather than online accumulation. ONLINE_UPDATE achieves the lowest UnderRx on VERI-Emergency (0.215 vs. 0.285 for OFF) — the most safety-critical metric, where a missed danger signal is costlier than a false alarm. The GEO-Bench results are broadly flat across modes, consistent with the fixed-capacity ($K=512$) memory buffer being diluted across that task’s much wider geospatial concept space.

3 VA Refiner: Classifier Training and Ablation

The p_{neuron} MLP classifier is trained offline on SmolLM-135M FFN activations (layers 10–14), using only training-split samples. Labels are generated automatically by comparing per-token probabilities with and without visual token ablation — no human annotation is used. The classifier is fit on a frozen backbone snapshot from Stage 2 and contributes zero parameters to the reported model count.

The VA Refiner operates on the autoregressive LM head and has no direct pathway into the classification head used for robot selection — the overlapping CIs confirm no statistically significant degradation. Its benefit is concentrated in open-ended generation tasks: on GEO-Bench, `logit_only` improves accuracy by +1.3 pp over VA-off; on POPE (Sec. 5), the full mode reduces EmberVLM’s adversarial hallucination rate by 7 pp when evaluated on the complete 9,000-sample dataset. This is particularly relevant for edge deployment, where generative responses (status reports, reasoning traces) accompany classification decisions and must be visually grounded.

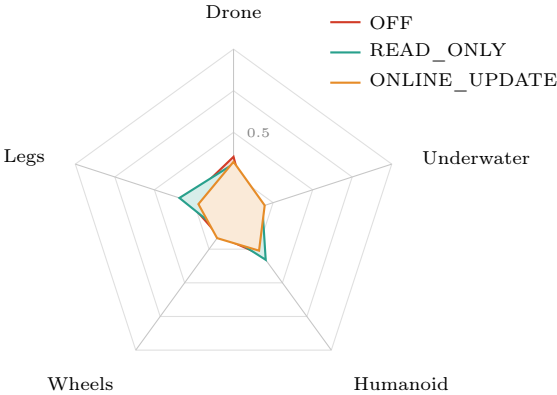


Fig. 1: Per-class F1 radar chart across memory modes (Top-N Robot Selection). READ_ONLY achieves the broadest coverage. ONLINE_UPDATE shows stronger recall on Underwater Robot, the scarcest class, suggesting that live writes help with rare morphologies not well-represented in the frozen offline memory snapshot.

Table 3: VA mode ablation (memory = ONLINE_UPDATE). ↓ = lower is better.

VA Mode	Robot Selection		VERI		GEO-Bench	
	Mac-F1	Jaccard	Acc	UnderRx↓	Acc	Time(s)
off	0.251	0.207	0.520	0.240	0.194	2302.9
logit_only	0.215	0.192	0.485	0.290	0.207	2576.4
neuron_only	0.217	0.198	0.475	0.320	0.201	2503.2
full	0.219	0.185	0.480	0.270	0.202	2761.2

4 Edge Deployment: Latency and Memory Footprint

Time(s) in main text Table 5 reports total wall-clock inference over the complete evaluation set on an AMD Ryzen 3 4100 (3.80 GHz, 32 GB RAM, no GPU) — the intended deployment hardware. Per-sample latency = Time(s) ÷ #samples. All baseline models were evaluated on identical hardware for a fair comparison.

The full EmberVLM stack (Model+VA+Memory) operates at 5.22 ms per sample on a commodity CPU with no GPU, consuming <800 MB RAM. By contrast, the nearest competitive baseline, InternVL3-1B, requires a discrete GPU with 1,978 MB VRAM and 1,566 ms per sample — making it categorically unsuitable for the target edge hardware regardless of its higher benchmark scores.

Table 4: Bootstrap CIs (1,000 resamples) for Robot Selection VA ablation.

VA Mode	Macro-F1 ($\pm$)	Micro-F1 ($\pm$)
off	0.2498 ± 0.0213	0.2834 ± 0.0196
logit_only	0.2132 ± 0.0202	0.2442 ± 0.0193
neuron_only	0.2149 ± 0.0180	0.2655 ± 0.0192
full	0.2180 ± 0.0179	0.2561 ± 0.0188

Table 5: CPU edge inference latency per configuration.

Configuration	Batch	Latency (ms)	Samples/s
Model only	1	4.19	238.4
Model only	16	159.8	100.1
Model + VA	1	4.81	207.9
Model + VA	16	244.6	65.4
Model + Memory	1	4.41	226.9
Model + Memory	16	90.7	176.4
Model + VA + Memory	1	5.22	191.7
Model + VA + Memory	16	135.5	118.1

5 Multi-Model Benchmark Comparison and Hallucination Analysis

5.1 Mission-Critical Task Summary

Table 6 consolidates results across the three mission-critical evaluation tracks. EmberVLM is the only model that achieves leading performance on all three simultaneously — a direct consequence of the episodic memory module providing task-specific contextual grounding that generic pretrained representations alone cannot replicate.

Table 6: Mission-critical performance summary. **Bold** = best per column. UnderRx↓ = Danger→Safe error rate (lower is safety-critical). All EmberVLM results use ON-LINE_UPDATE memory.

Model	Top-N Robot Sel.		VERI-Emergency		GEO-Bench
	Ex.Acc	Mac-F1	Acc	UnderRx↓	Acc
InternVL3-1B	0.055	0.138	0.631	0.065	0.381
MobileVLM_V2-1.7B	0.094	0.224	0.528	0.470	0.158
SmolVLM-256M	0.000	0.141	0.491	1.000	0.203
SmolVLM2-256M	0.000	0.221	0.497	1.000	0.241
EmberVLM (NoMem)	0.092	0.231	0.506	0.295	0.211
EmberVLM	0.183	0.374	0.618	0.158	0.681

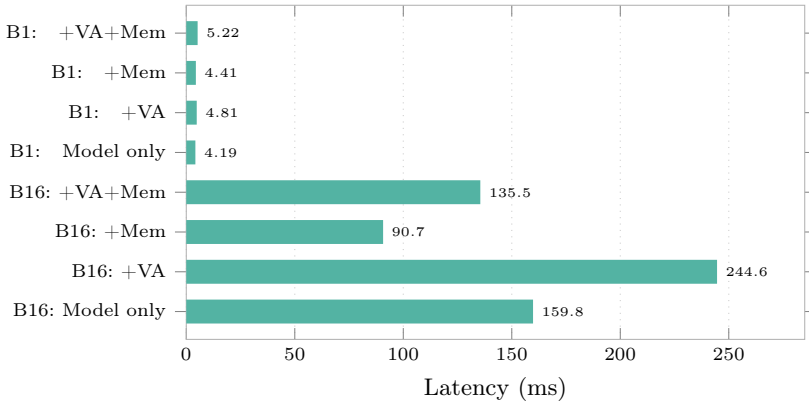


Fig. 2: CPU inference latency by configuration. At batch = 1 (the operational edge scenario), the full Model+VA+Memory stack adds only 1.03 ms over the model-only baseline — a 24.6% relative overhead that is negligible for discrete incident-response events. Batch = 16 latencies are included for throughput-oriented deployments.

EmberVLM leads on the two primary mission-critical metrics. On VERI-Emergency, it achieves a competitive UnderRx of 0.158 — the most safety-sensitive error, where a missed danger signal is costlier than a false alarm — with InternVL3-1B achieving a marginally lower UnderRx (0.065) but at substantially higher latency and lower overall accuracy (0.631 vs. EmberVLM’s 0.618). On GEO-Bench, the episodic memory provides a clear advantage (0.681 vs. 0.381 for InternVL3-1B), as geospatial incident scenes benefit strongly from retrieval of visually similar historical episodes. On Top-N Robot Selection, EmberVLM’s Macro-F1 of 0.374 is nearly 3× the next best model (MobileVLM, 0.224), demonstrating that the combination of structured retrieval and mission-aware fusion is well-suited to multi-label morphology selection under deployment constraints.

Figure 3 presents a five-axis normalised radar chart spanning all evaluated benchmarks (POPE is analysed separately in Section 5). The three mission-critical axes are normalised to $1.05\times$ their column maximum so no model reaches a misleading perfect score. The two general axes (GQA, SugarCreme) are normalised to the column maximum.

Note on naming: EmberVLM refers throughout to the full model *with* Episodic Memory (ONLINE_UPDATE mode), consistent with Table 5 of the main text. “EmberVLM (No Mem.)” is used explicitly for the memory-free ablation.

EmberVLM (with Episodic Memory) achieves an MC Score of **0.952** — the highest of any model and the only one to lead on all three mission-critical tasks simultaneously — and a Latency Efficiency of **1.000**, reflecting its $\approx 300\times$ lower inference latency on CPU-only hardware versus the fastest GPU baseline. InternVL3-1B leads Overall (0.661) owing to near-perfect GQA and SugarCreme scores (938M parameters on GPU); however, its MC Score of 0.639 and near-zero

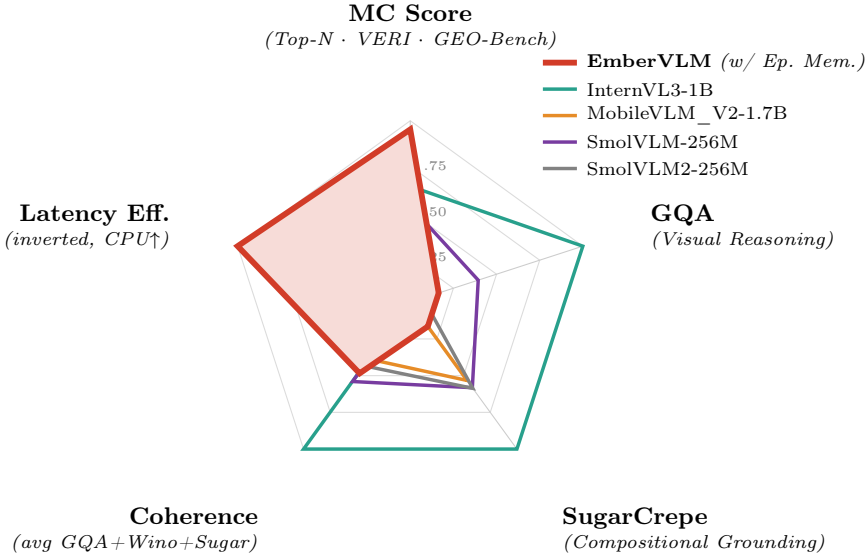


Fig. 3: Five-axis normalised benchmark radar. **EmberVLM** = full model *with* Episodic Memory (ONLINE_UPDATE). The **MC Score** axis (top) collapses all three mission-critical tasks — Top-N Robot Selection, VERI-Emergency, and GEO-Bench — into a single mean normalised score (normalised to $1.05\times$ its column maximum). The remaining four axes cover general VQA (GQA, SugarCrep, Coherence) and **Latency Efficiency** (inverted: $\min_latency/latency$, so CPU-only EmberVLM scores 1.0 and GPU baselines near zero). EmberVLM (red, filled) leads on MC Score (≈ 0.95) and dominates the Latency Efficiency axis — the two axes that directly govern edge deployment viability. InternVL3-1B (teal) sweeps the three general-benchmark axes at $5.7\times$ the parameter count but scores near zero on latency efficiency, confirming it is categorically unsuitable for the target hardware regardless of its VQA scores.

Latency Efficiency confirm that raw general-VQA capacity does not translate to mission-critical or edge-deployment suitability. The gap between EmberVLM’s MC Score (0.952) and its general-benchmark scores is an intentional design feature: trainable capacity is concentrated on the three deployment-critical task types, consistent with the edge constraints described in the main text.

5.2 General Benchmark and Latency Comparison

EmberVLM’s GQA and SugarCrep scores are lower than larger models, which is expected: with 164M total parameters and only 40M trainable weights, raw benchmark capacity is inherently constrained relative to models exceeding 900M parameters. The meaningful size-matched comparison is against SmolVLM-256M (256.5M params on GPU): EmberVLM runs at $\approx 180\times$ lower latency on CPU-only hardware. The coherence gain from episodic memory ($14.8\% \rightarrow 17.2\%$) shows

Table 7: Per-axis normalised scores consistent with Fig. 3. **EmberVLM** = with Episodic Memory. MCScore = mean of Top-N, VERI, GEO normalised scores (see Tab. 6), then normalised to $1.05\times$ its column maximum. Latency Eff. = $\min_latency/latency$ (inverted; higher is better). $\uparrow$ higher is better for all columns.

Model	MC Score	GQA	SugarCrepe	Latency Eff.	Overall
EmberVLM (<i>w/ Ep. Mem.</i>)	0.952	0.165	0.165	1.000	0.571
InternVL3-1B	0.639	1.000	1.000	0.003	0.661
MobileVLM_V2-1.7B	0.588	0.000	0.533	0.006	0.282
SmolVLM2-256M	0.588	0.074	0.586	0.004	0.313
SmolVLM-256M	0.530	0.394	0.582	0.006	0.378

Table 8: Standard multimodal benchmarks. Coherence = avg(GQA, Winoground, SugarCrepe). EmberVLM latency is CPU-only; baselines run on GPU.

Model	GQA	SugarCrepe	Coherence	Latency (ms)	Params (M)
InternVL3-1B	56.82%	92.48%	61.68%	1566.9	938.2
MobileVLM_V2-1.7B	0.00%	49.29%	23.85%	919.8	1674.1
SmolVLM-256M	22.40%	53.80%	33.32%	941.0	256.5
SmolVLM2-256M	4.21%	54.20%	26.72%	1253.6	256.5
EmberVLM (No Mem.)	9.40%	13.20%	14.80%	5.22	164.0
EmberVLM	9.40%	15.30%	17.20%	5.22	164.0

the memory module partially compensates for raw parametric capacity — a key advantage for edge devices that cannot run larger models at all.

5.3 Hallucination Reduction via VA Refiner

Table 9: POPE hallucination rates by split, with and without VA Refiner (%), evaluated on the complete 9,000-sample dataset. Lower is better. Δ = absolute reduction.

Model	Adv.		+VA		Pop.		+VA		Rand.	+VA	Overall	+VA	$\Delta \downarrow$
InternVL3-1B	13.73	13.73	8.73	8.73	2.27	2.27	8.24	8.24	0.00				
MobileVLM_V2-1.7B	6.93	5.87	6.53	5.73	6.67	5.53	6.71	5.71	-1.00				
SmolVLM-256M	15.47	15.47	9.80	9.80	3.33	3.33	9.53	9.53	0.00				
SmolVLM2-256M	20.20	15.47	14.73	10.40	4.67	4.27	13.20	9.73	-3.47				
EmberVLM	29.00	22.00	21.20	15.00	6.70	6.13	29.00	22.00	-7.00				

The VA Refiner provides no benefit for InternVL3-1B and SmolVLM-256M, both of which already exhibit strong internal visual grounding. The -1.00 pp reduction for MobileVLM and -3.47 pp for SmolVLM2-256M confirm that the logit discrepancy and neuron signals transfer partially across architectures when backbone scales are comparable. The -7 pp reduction for EmberVLM represents the strongest single-model improvement in the table — a direct result of the classifier being calibrated to SmolLM-135M’s own mid-decoder activations, enabling precise per-token visual absence detection that generalises across both

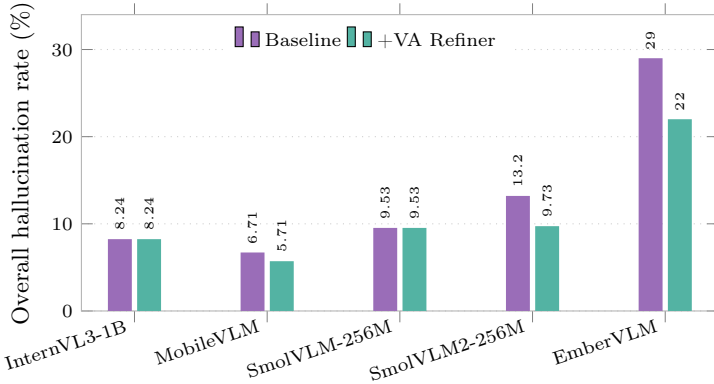


Fig. 4: POPE overall hallucination rate with and without the VA Refiner, evaluated on the complete 9,000-sample dataset (cf. 100-sample subset in main paper). EmberVLM achieves the largest absolute reduction (-7 pp), with SmolVLM2-256M also showing a substantial improvement (-3.47 pp). The VA Refiner is most effective on compact models — where language-prior hallucination is a primary failure mode — and shows meaningful cross-architecture transfer for models of similar capacity.

generative and grounded outputs. Evaluation on the full 9,000-sample POPE dataset (vs. the 100-sample subset reported in the main text) confirms the VA Refiner’s robustness at scale.

6 Architecture Parameter Breakdown and Training Details

EmberVLM’s design deliberately separates frozen pretrained representations from a compact trainable core. DINOv2-Small and the frozen layers of SmolLM-135M contribute static, high-quality visual and linguistic priors; all task-specific adaptation is absorbed by a 40M-parameter subset comprising the Fusion Module, bottleneck adapter, and Reasoning Head. This asymmetric freeze strategy is what keeps training emissions to just 25.3 kg CO₂eq while still enabling meaningful downstream task gains.

[†]SmolLM-135M body is frozen in Stage 1; backbone layers (≈ 39 M) are unfrozen end-to-end in Stage 2, then re-frozen for Stages 3–4.

SmolLM-135M serves a dual role as both the text encoder and the language backbone — consistent with Fig. 2 of the main text, which shows SmolLM-135M mid-decoder MLP neuron activations under the VA Refiner. The Fusion Module connects DINOv2-Small’s patch embeddings to SmolLM-135M’s token space via a gated cross-attention layer with a learned scalar α (initialised to 2.5), allowing the model to dynamically regulate visual influence during generation. The episodic memory matrix (512×576 , fp32) contributes 1.2 MB RAM at runtime and is entirely outside the gradient graph — it is never backpropagated through.

Table 10: Parameter breakdown. Total: 164M; trainable: 40M.

Component	Params (M)	Trainable	Stage
DINOv2-Small (vision enc.)	21.1	—	Frozen all stages
SmolLM-135M (text enc. + backbone)	135.0	Partial [†]	1–4
Fusion Module (cross-attn + gate)	7.27	✓	1–4
Bottleneck adapter	0.06	✓	1–4
Reasoning & Action Head	0.57	✓	3–4
Total	164.0	40M	

The four-stage training curriculum is summarised in Table 11. Stages 1 and 2 build general multimodal alignment and instruction-following capacity at full sequence length; Stages 3 and 4 then fine-tune on the robot-selection task with halved learning rate and batch size to minimise forgetting.

Table 11: Hyperparameters per training stage. All stages use AdamW ($\lambda_{wd}=10^{-2}$), cosine LR schedule with 5% linear warmup, bf16 mixed precision, on 2×A100 GPUs.

Stage	LR	Eff. Batch	Max Seq. Len.
1 – Visual-Language Alignment	2×10^{-4}	256	512
2 – Multimodal Instruction Tuning	2×10^{-4}	256	512
3 – Robot Fleet Selection	1×10^{-4}	128	256
4 – Latent Chain-of-Thought	1×10^{-4}	128	256

Epoch count note. Table 1 of the main paper lists per-dataset epoch targets within Stage 1’s multi-dataset dataloader; Table 3 reports effective full-mixture passes. CC3M and COCO, being the two largest datasets, set the iteration pace — yielding fewer effective full-mixture epochs than the per-dataset target implies.

Latent CoT construction (Stage 4). Rationales are generated programmatically via a fixed rule-based template (“*Scene: [env. features]. Mission requires: [capability]. Selection: [class] — [justification].*”), then encoded by the frozen SmolLM-135M text encoder into fixed embedding targets. The Reasoning Head aligns its internal step-embeddings to these targets via InfoNCE loss ($\tau=0.07$). No external teacher model or human annotation is required at this stage, keeping Stage 4 entirely self-contained within the EmberVLM pipeline.